

Databases & Architecture — Placement Essentials

Most important concepts and questions for placement interviews

DBMS Fundamentals

1. What is the difference between DBMS and RDBMS?

DBMS manages data as files with no relational structure enforced; RDBMS stores data in related tables, enforces schema, and supports relationships via keys and constraints.

2. What are primary key, foreign key, and candidate key?

Primary key uniquely identifies a row and can't be null. Candidate key is any column set that could serve as a primary key. Foreign key references a primary key in another table to enforce referential integrity.

3. What is normalization? Name the normal forms up to 3NF.

Normalization organizes data to reduce redundancy. 1NF: atomic columns, no repeating groups. 2NF: 1NF + no partial dependency on a composite key. 3NF: 2NF + no transitive dependency on non-key attributes.

4. What is denormalization and when is it used?

Deliberately introducing redundancy (merging tables, duplicating data) to reduce joins and speed up reads — used in read-heavy, reporting, or analytics systems.

5. What are the different types of joins?

INNER JOIN returns matching rows only; LEFT/RIGHT JOIN returns all rows from one side plus matches; FULL OUTER JOIN returns all rows from both sides; CROSS JOIN returns the Cartesian product.

6. What is a database index, and what's the trade-off of using one?

An index is a data structure (typically B-tree) that speeds up lookups on a column, avoiding full table scans. Trade-off: faster reads but slower writes (index must be updated) and extra storage.

Transactions & Concurrency

1. What are the ACID properties?

Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives failures).

2. What are the transaction isolation levels?

Read Uncommitted, Read Committed, Repeatable Read, Serializable — increasing strictness, trading concurrency/performance for consistency; they prevent dirty reads, non-repeatable reads, and phantom reads to varying degrees.

3. What is a deadlock, and how can it be prevented?

A deadlock occurs when two transactions each hold a lock the other needs, blocking both indefinitely. Prevention: consistent lock ordering, timeouts, deadlock detection with automatic rollback.

4. What is the difference between optimistic and pessimistic locking?

Pessimistic locking locks a resource upfront to prevent conflicts, blocking other transactions. Optimistic locking allows concurrent access and checks for conflicts (e.g. via a version number) only at commit time.

5. What is a database trigger?

A stored procedure that automatically executes in response to a specific event (INSERT/UPDATE/DELETE) on a table, often used for auditing, validation, or maintaining derived data.

SQL vs NoSQL

1. What is the difference between SQL and NoSQL databases?

SQL (Postgres, MySQL) is relational, schema-enforced, ACID-compliant, and scales vertically. NoSQL (MongoDB, Cassandra, Redis) is schema-flexible, often trades strict consistency for availability/scale, and scales horizontally more naturally.

2. What are the different types of NoSQL databases?

Document stores (MongoDB), key-value stores (Redis, DynamoDB), column-family stores (Cassandra), and graph databases (Neo4j) — each suited to different access patterns.

3. When would you choose NoSQL over SQL?

When schema flexibility is needed, data volume/velocity is very high, horizontal scaling is a priority, or the access pattern doesn't need complex joins/transactions — e.g. session storage, event logs, real-time feeds.

4. What is the CAP theorem?

In a distributed system, you can only guarantee two of Consistency, Availability, and Partition tolerance at once. Since network partitions are unavoidable in practice, real systems choose between CP (consistent but may reject requests) and AP (available but may return stale data).

Scaling & Distributed Architecture

1. What is database replication?

Copying data across multiple database nodes (typically one primary, multiple replicas) to improve read throughput and provide failover/availability if the primary fails.

2. What is database sharding?

Horizontally partitioning data across multiple database instances (shards) based on a key, so each shard holds a subset of data — used to scale writes and storage beyond a single machine's limits.

3. What is the difference between horizontal and vertical scaling?

Vertical scaling adds more resources (CPU/RAM) to a single server — simple but has a hardware ceiling. Horizontal scaling adds more servers/nodes — more complex (requires statelessness/data partitioning) but scales further.

4. What is a load balancer, and what algorithms does it use?

A component that distributes incoming traffic across multiple servers to improve availability and throughput, using strategies like round-robin, least connections, IP hash, or weighted distribution.

5. What is caching, and where can it be applied in a system?

Storing frequently accessed data in fast storage to reduce latency and database load. Can be applied at the client, CDN, application (e.g. Redis/Memcached), or database query-result level, with strategies like cache-aside, write-through, and TTL-based invalidation.

6. What is the difference between monolithic and microservices architecture?

A monolith is a single deployable unit containing all functionality — simpler to build/deploy but harder to scale specific parts independently. Microservices split functionality into independently deployable services — better scalability/isolation but add network overhead and operational complexity.

7. What is a message queue, and why is it used in system architecture?

A component (e.g. Kafka, RabbitMQ, SQS) that decouples producers and consumers of data, enabling asynchronous processing, buffering traffic spikes, and improving fault tolerance between services.

8. What is eventual consistency?

A consistency model where, given enough time without new updates, all replicas of data will converge to the same value — used in distributed/NoSQL systems that prioritize availability over immediate consistency.